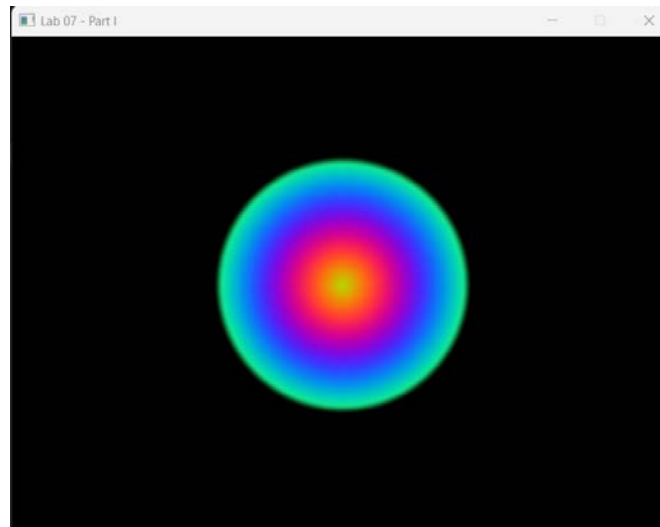


Lab 07: Shaders

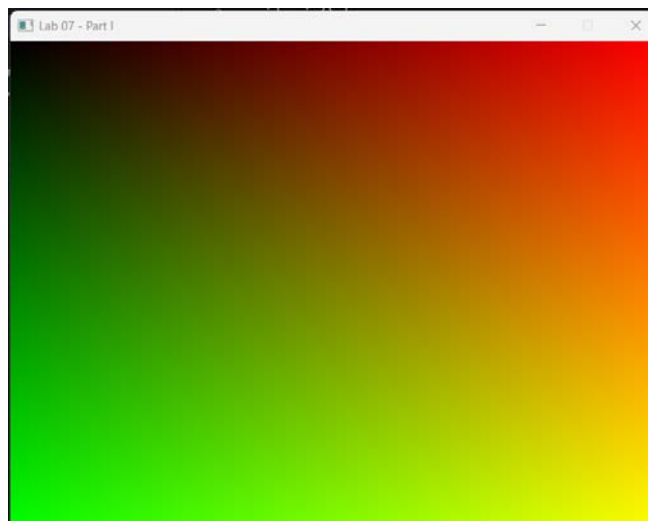
Part I:

```
shaders    fullscreen.vert + circle.frag, no vertex buffer
uniform    176 bytes, time 1.000 s, mode 0
coverage   15.9% of the target,  $\pi * 0.25 / (2 * 8/3) = 14.7\%$  before the AA ring
centre     pixel (400, 300) reads (42, 77, 255), cpu says (42, 77, 255)
mode 1     (0, 0) reads (0, 0, 0), expected (0, 0, 0)
mode 1     (799, 0) reads (255, 0, 0), expected (255, 0, 0)
mode 1     (0, 599) reads (0, 255, 0), expected (0, 255, 0)
image      MATCH (0.00% differ)
gpu time   0.166 ms median, 0.231 ms p95
keys       0..3 debug modes, Up/Down octaves, S saves a png, Esc quits
watching   D:/Engineering/Year_III/Computer_Graphics/assignments/lab7/2110479-Computer-Graphics-Lab07-Shaders/shaders
```

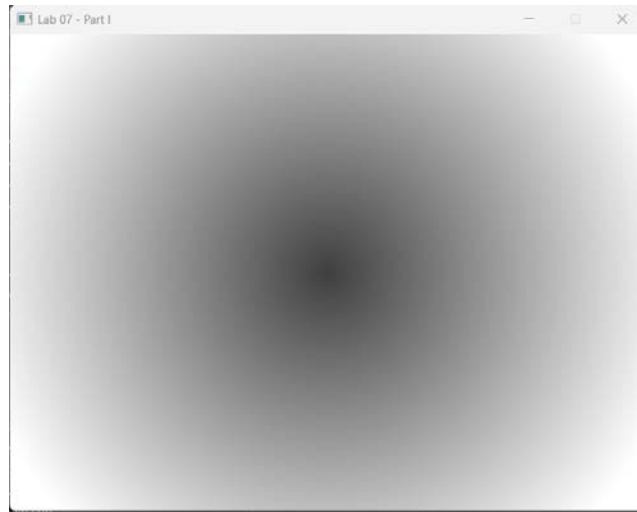
Mode: 0



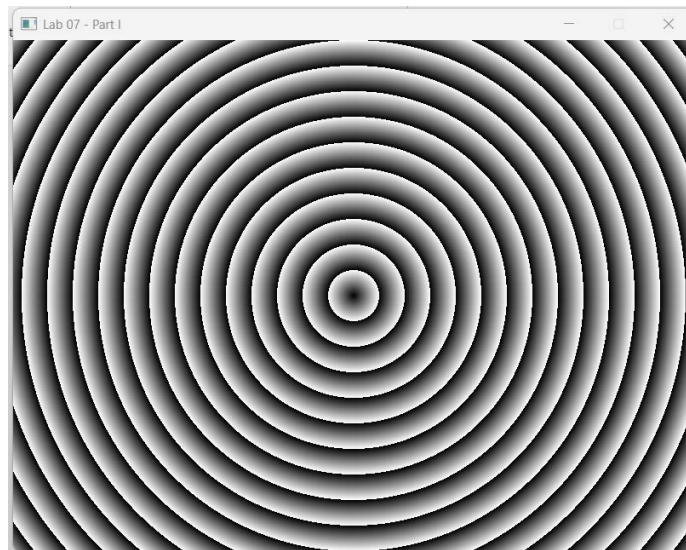
Mode: 1



Mode: 2



Mode: 3



Write-ups

1. Where the y flip that Lab 05's viewportTransform() used to do has gone, which corner of your mode 1 view is green, and the one line you would have to change in a shader copied from ShaderToy.

Ans In Lab 05, viewportTransform() explicitly changed NDC's upward-pointing y-axis into the browser's downward-increasing pixel rows using $y = (1 - \text{ndc.y}) / 2 \times \text{height}$. In Lab 07, that flip is handled by Vulkan's viewport and framebuffer-coordinate convention, so gl_FragCoord.y already increases downward. Therefore, in mode 1, green increases toward the bottom and the bottom-left corner is green. ShaderToy uses a bottom-left origin, so a copied ShaderToy shader must flip y once with:

```
vec2 fragCoord = vec2(gl_FragCoord.x, u.resolution.y -  
gl_FragCoord.y);
```

2. Why a fixed threshold, smoothstep(0.01, -0.01, d), is resolution dependent while smoothstep(w, -w, d) is not. Give the width of each, in pixels, at 800×600 and at 3840×2160 .

Ans Smoothstep(0.01, -0.01, d) is resolution dependent because 0.01 is measured in p-coordinate units rather than pixels. The complete transition covers 0.02 p units. At 800×600 , one p unit represents 300 pixels, so the transition is $0.02 \times 300 = 6$ pixels wide. At 3840×2160 , one p unit represents 1080 pixels, so it becomes $0.02 \times 1080 = 21.6$ pixels wide. In contrast, when $w = \text{fwidth}(d)$, w changes according to the amount that d varies between neighboring pixels. Therefore, smoothstep(w, -w, d) maintains an approximately 1-pixel-scale transition at both 800×600 and 3840×2160 .

3. What fwidth(d) is, and how the GPU can take a derivative of a value that each pixel computed for itself. Name the hardware fact, and the Lab 06 part where you measured it.

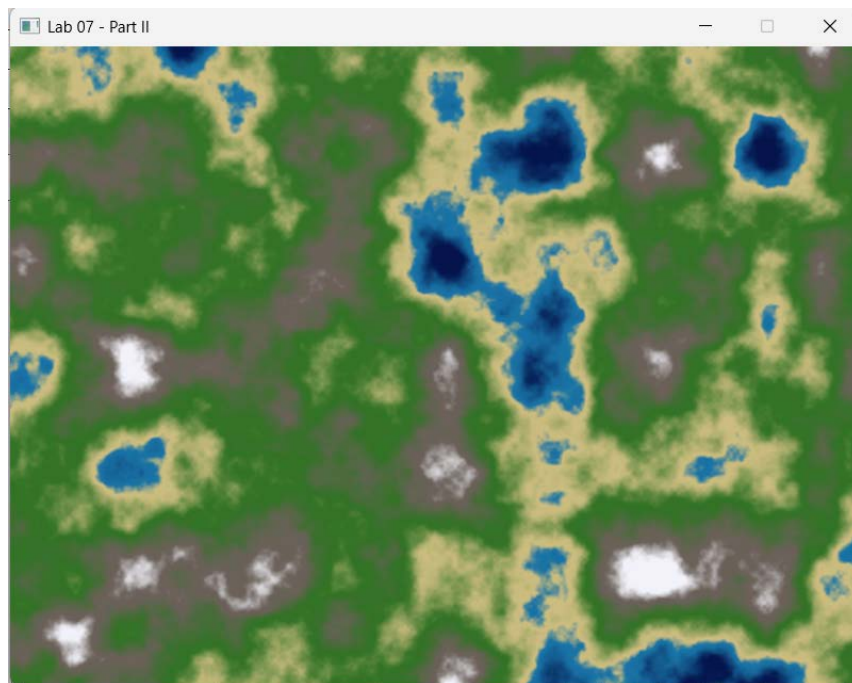
Ans fwidth(d) is $\text{abs}(\text{partial } d \text{ by partial } x) + \text{abs}(\text{partial } d \text{ by partial } y)$. It estimates how much d changes between neighboring fragments in the horizontal and vertical directions. The GPU can calculate this even though each fragment computes its own d because fragment shaders execute in groups containing 2×2 pixel quads. The GPU compares the d values produced by neighboring invocations in a quad to estimate the derivatives. The relevant hardware fact is that neighboring fragment invocations execute together rather than independently. This behavior was investigated in Lab 06 Part III using the stripe-width branch-divergence experiment. That experiment measured subgroup divergence and the grouping of neighboring fragments, although it did not directly measure fwidth itself.

Part II:

```

shaders      fullscreen.vert + terrain.frag, which includes noise.glsl
uniform      time 1.000 s, mode 0, octaves 6
lattice      pixel (300, 200) is p = (3, 2): value_noise reads 169, cpu hash(uvec2(1003, 1002)) is 169
one octave   fbm(p, 1) reads 84, half of 169 is 84
fbm ceiling  1 - 0.5^8 = 0.996, brightest pixel of the image 0.914
octaves 1    0.558 ms median
octaves 2    0.859 ms median
octaves 3    1.146 ms median
octaves 4    1.440 ms median
octaves 5    1.755 ms median
octaves 6    2.052 ms median
octaves 7    2.379 ms median
octaves 8    2.640 ms median
image        MATCH (0.00% differ) at 6 octaves
gpu time     2.125 ms median, 4.916 ms p95
keys         0..3 debug modes, Up/Down octaves, S saves a png, Esc quits
watching     D:/Engineering/Year_III/Computer_Graphics/assignments/lab7/2110479-Computer-Graphics-Lab07-Shaders/shaders
reloaded     fullscreen.vert
reloaded     terrain.frag

```

Write-ups

1. Before measuring: your predicted ratio between 8 octaves and 1, the shape of the curve, and one line of reasoning.

Ans I think that 8 octaves would take approximately 8 times as long as 1 octave and that the graph would be linear. Each additional octave performs another `value_noise()` evaluation with approximately the same instruction cost, even though its amplitude becomes smaller.

2. Your eight measurements, the ms per octave, and whether the curve is linear. Compare with your prediction. If it is not linear, say what else the measurement is measuring.

Ans My measurements were 0.631 ms for 1 octave, 0.879 ms for 2, 1.229 ms for 3, 1.489 ms for 4, 1.815 ms for 5, 2.113 ms for 6, 2.400 ms for 7, and 2.725 ms for 8. The fitted cost was approximately 0.300 ms per additional octave, with about 0.308 ms of fixed overhead. The curve was approximately linear, but 8 octaves were only $2.725 / 0.631 = 4.32$ times slower than 1 octave, not the predicted 8 times. This is because the measurement also includes fixed costs such as rasterization, framebuffer operations, pipeline work, and timing overhead that do not increase with the octave count.

3. The budget in section 7.6's scenario is 0.2 ms. How many octaves can you afford at 800×600 , and how many at 4K, which is $17.3\times$ the pixels? State the assumption your extrapolation makes.

Ans With a cost of approximately 0.300 ms per octave, a 0.2 ms budget cannot afford even 1 octave at 800×600 , so the answer is 0 octaves. At 4K, the estimated cost per octave is $0.300 \times 17.3 = 5.19$ ms, so the budget still permits 0 octaves. This extrapolation assumes execution time scales linearly with pixel count and that GPU frequency, memory behavior, and other overheads remain unchanged.

4. Why your friend's GPU draws a different terrain when the hash is the chapter's $\text{fract}(\sin(\text{dot}(p, \dots)) * 43758.5453)$, and why the integer hash does not have that problem.

Ans The expression $\text{fract}(\sin(\text{dot}(p, \dots)) * 43758.5453)$ uses the sine of a large floating-point value. GPU vendors may implement sin

with different approximations and precision, so small differences become large after multiplication and fract, producing different terrain. The PCG hash uses unsigned integer multiplication, shifts, and XOR operations. These operations have precisely defined results, so the same input produces the same hash on different GPUs.

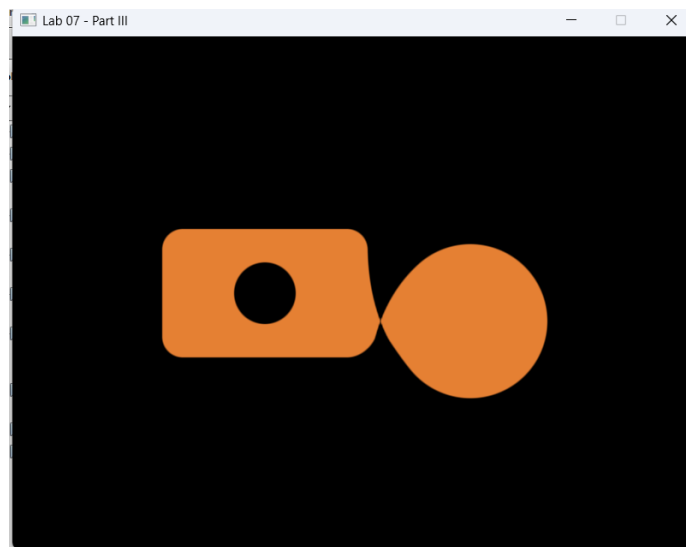
5. Why Hermite $f^2(3 - 2f)$ and not plain f . Swap them, look at mode 2, and describe what appears.

Ans The Hermite weight $f \times f \times (3 - 2 \times f)$ has a derivative of 0 at both ends of a cell. This allows adjacent noise cells to join smoothly. Using plain f keeps the noise values continuous, but its derivative changes abruptly at lattice boundaries. In mode 2, this produces visible square, grid-aligned creases and makes the individual noise cells easier to see.

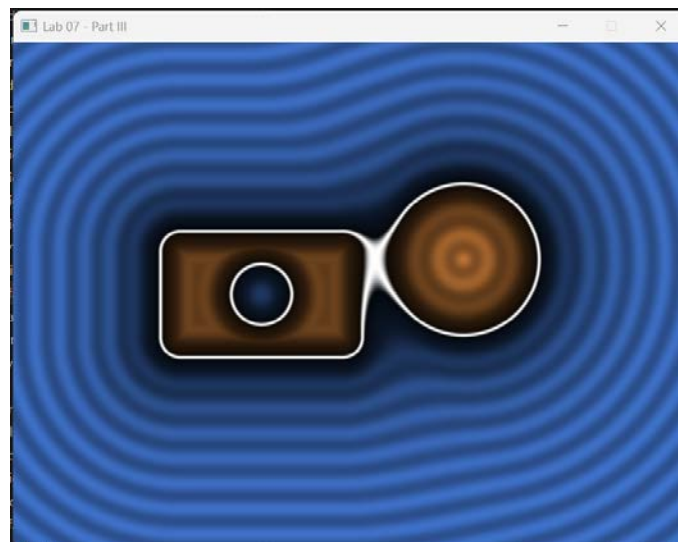
Part III:

```
shaders      fullscreen.vert + sdf.frag
uniform      time 1.000 s, mode 0
probe        (220, 300) inside the body      shader d -0.129  cpu d -0.128  inside
probe        ( 60,  60) outside everything    shader d  0.702  cpu d  0.700  outside
probe        (430, 300) in the blend          shader d  0.012  cpu d  0.010  outside
probe        (295, 300) inside the hole       shader d  0.114  cpu d  0.118  outside
bound        in the blend the field says 0.010, the nearer primitive is
              0.052 away: after a smooth union d is a bound, not a distance
coverage      12.5% of the target is shape, the rest is exactly black
image        MATCH (0.00% differ)
gpu time      0.531 ms median, 0.534 ms p95
keys          0..3 debug modes, Up/Down octaves, S saves a png, Esc quits
              hold the left mouse button in mode 0 to drag the ball
watching      D:/Engineering/Year III/Computer Graphics/assignments/lab7/2110479-Computer-Graphics-Lab07-Shaders/shaders
```

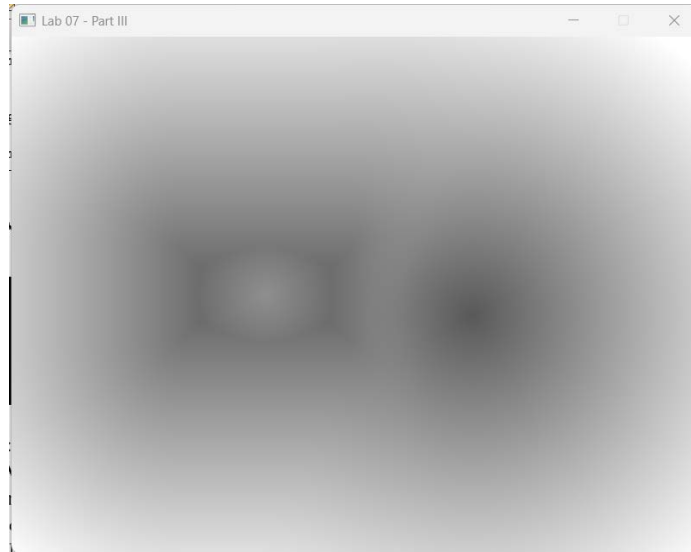
Mode: 0



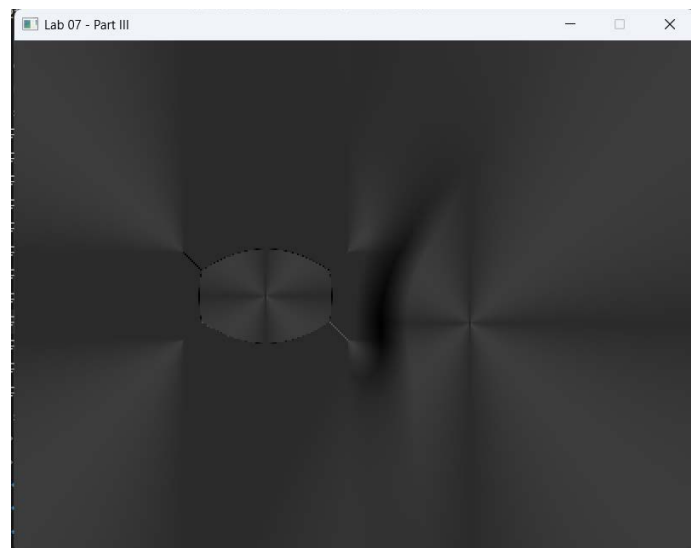
Mode: 1



Mode: 2



Mode: 3



Write-ups

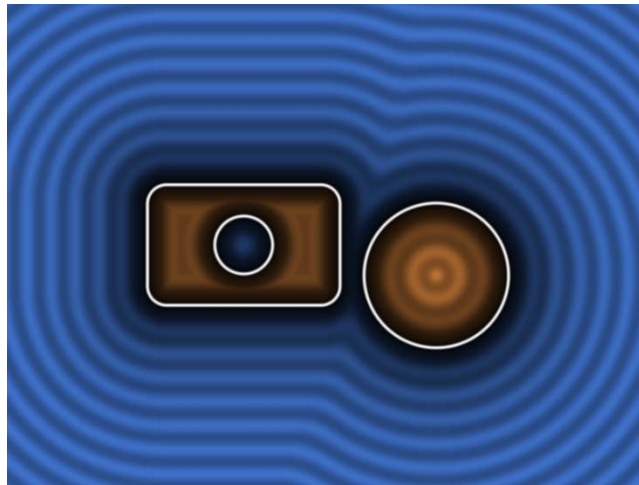
1. Why $\min$ is union and $\max(d1, -d2)$ is subtraction. One sentence each, in terms of what the distance to the combined shape must be.

Ans The union uses $\min(d1, d2)$ because a point belongs to the combined shape when it is inside either shape, and the smaller signed distance selects that shape. Subtraction uses $\max(d1, -d2)$ because

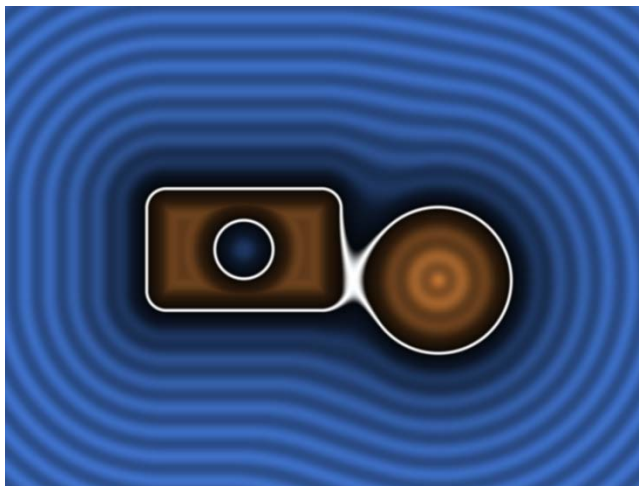
negating d_2 reverses the inside and outside of the removed shape, while $\max$ keeps a point inside only when it is inside the first shape and outside the removed shape.

2. What k does to the blend, and what it does to the contour lines in mode 1. Change it, screenshot two values, and say which one you would ship.

Ans The value k controls the width and strength of the smooth union. A smaller k approaches a hard minimum, while a larger k creates a wider connecting region and bends the contour lines farther away from the junction.



With $k = 0.05$ the shapes remain visually separate, the contours mostly follow the original primitives, and coverage is 12.2%.



With $k = 0.20$, a narrow smooth connection appears, the contours bend and bunch around the junction, and coverage becomes 12.5%.

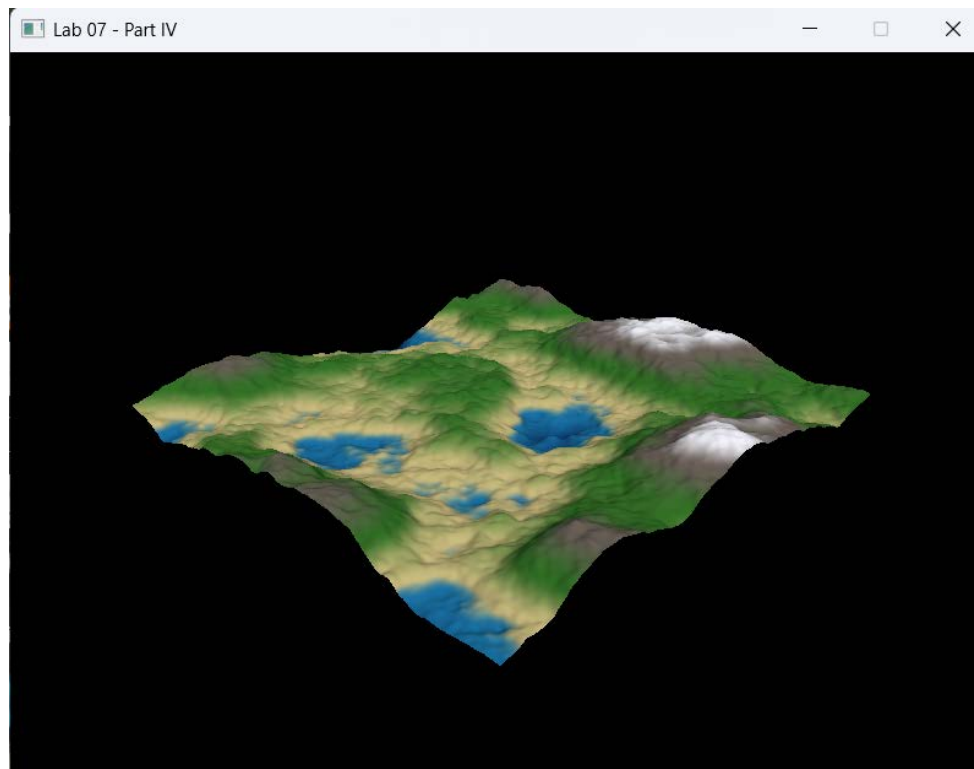
I would ship $k = 0.20$ because it clearly joins the shapes without creating an excessively large bulge.

3. The field is the exact distance for a circle but only a bound after a smooth union. Quote your blend probe, say by how much it underestimates, and name one algorithm that would break because of it.

Ans At the blend probe, the field reports 0.010 while the nearer original primitive is 0.052 away. Therefore, the field underestimates the distance by $0.052 - 0.010 = 0.042$ units, or approximately 80.8%. The shader displays 0.012 because the debug value is quantized into an 8-bit color channel. After the smooth union, the sign still identifies the inside and outside correctly, but the magnitude is only a bound rather than an exact distance. A sphere-tracing ray marcher that assumes a true SDF could take unnecessarily small steps and exhaust its fixed step limit before reaching the surface. The shader remains restored to $k = 0.20$ and passes the reference comparison with 0.00% difference.

Part IV:

```
shaders      heightmap.vert + heightmap.frag, the vertex stage does fbm
grid         N 256: N*N = 65536 vertices, 6*(N-1)^2 = 390150 indices, 130050 triangles
camera       eye (2.2, 1.6, 2.2), 45 deg, reversed-Z: clear 0, compare GREATER
normals      mode 1 centre pixel (149, 244, 81), flat ground would read (128, 255, 128)
coverage     18.1% of the target, 86734 fragments
vertex stage N 64      0.217 ms median, 4096 vertices
vertex stage N 128     0.815 ms median, 16384 vertices
vertex stage N 256     2.756 ms median, 65536 vertices
vertex stage N 512    10.051 ms median, 262144 vertices
fragment stage N 64    0.764 ms median, 90772 fragments run the same fbm
image        MATCH (0.00% differ) at N 256
gpu time     2.814 ms median, 2.830 ms p95
keys         0..3 debug modes, Up/Down octaves, S saves a png, Esc quits
              [ and ] halve and double the grid, which rebuilds the mesh
watching     D:/Engineering/Year_III/Computer_Graphics/assignments/lab7/2110479-Computer-Graphics-Lab07-Shaders/
```



Write-ups

1. Before measuring: rank the five rows fastest to slowest, with one line of reasoning each in invocation counts.

Ans Before measuring, I predicted the following order from fastest to slowest: vertex N = 64, with 4,096 vertices and 12,288 fBM evaluations; vertex N = 128, with 16,384 vertices and 49,152

evaluations; fragment $N = 64$, with 90,772 fragment evaluations; vertex $N = 256$, with 65,536 vertices and 196,608 evaluations; and vertex $N = 512$, with 262,144 vertices and 786,432 evaluations. This ranking uses the handout's stated assumption of three height evaluations per vertex.

2. Your five rows, with the invocation count beside each, against your ranking. Explain the ratio between the $N = 64$ vertex row and the $N = 64$ fragment row using nothing but those counts and the three calls per vertex. Then explain why the vertex rows do not quite quadruple when N doubles.

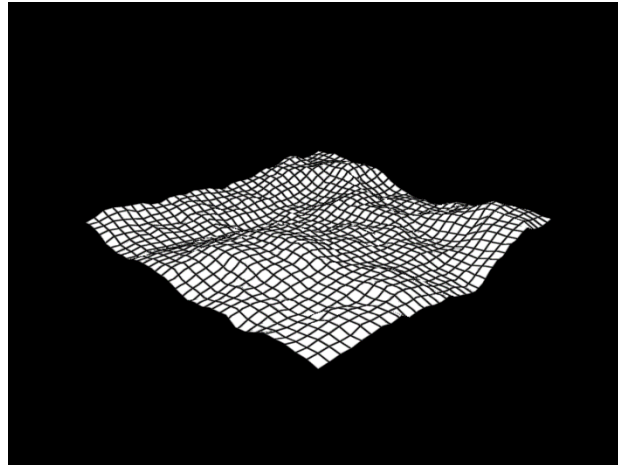
Ans My measurements were 0.220 ms for 4,096 vertices at $N = 64$, 0.830 ms for 16,384 vertices at $N = 128$, 2.809 ms for 65,536 vertices at $N = 256$, 10.337 ms for 262,144 vertices at $N = 512$, and 0.763 ms for 90,772 fragment invocations at $N = 64$. The measured order was vertex $N = 64$, fragment $N = 64$, vertex $N = 128$, vertex $N = 256$, and vertex $N = 512$. Using the handout's three-call accounting, $N = 64$ performs $4,096 \times 3 = 12,288$ fBM evaluations in the vertex stage, while the fragment version performs 90,772. The count ratio is $90,772 / 12,288 = 7.39$, while the measured time ratio is $0.763 / 0.220 = 3.47$. Fixed rendering costs and different vertex and fragment execution efficiencies prevent the timing ratio from matching the count exactly. When N doubles, the vertex count quadruples, but the measured ratios were 3.77, 3.38, and 3.68 rather than exactly 4. Fixed pipeline overhead, GPU parallelism, cache behavior, clock variation, and measurement noise prevent perfect scaling.

3. Why the finite difference normal needs three `height()` calls, and what $dFdx/dFdy$ of the world position in the fragment shader would have given you instead. Say what the picture would look like.

Ans The handout describes one height evaluation for the vertex position and two additional neighboring evaluations for the normal, giving three evaluations per vertex. However, its prescribed central-difference formula actually evaluates height at $x - e$, $x + e$, $z - e$, and $z + e$. Including the center height, the implemented shader therefore contains five height calls, not three. Three calls would only be enough for a forward-difference normal that reuses the center height and samples one neighbor in each direction. Calculating $dFdx$ and $dFdy$ from the interpolated world position in the fragment shader would instead produce the geometric normal of each rasterized triangle. Because the world position changes linearly across a triangle, its derivatives are constant there, so the terrain would look flat-shaded and faceted, with visible triangular faces.

4. What happens to the silhouette when N drops below the frequency of the noise. Press [until you can see it, screenshot it, and name what the same failure is called when it happens to a texture.

Ans At $N = 32$, the terrain does not contain enough vertices to sample the higher-frequency noise accurately. Peaks and valleys between vertices are missed, and the silhouette becomes angular and consists of straight line segments. The saved [$N = 32$ screenshots] shows the coarse grid and segmented outline. The equivalent texture-sampling failure is spatial aliasing or undersampling; during movement it can also appear as shimmering or popping.



Part V:

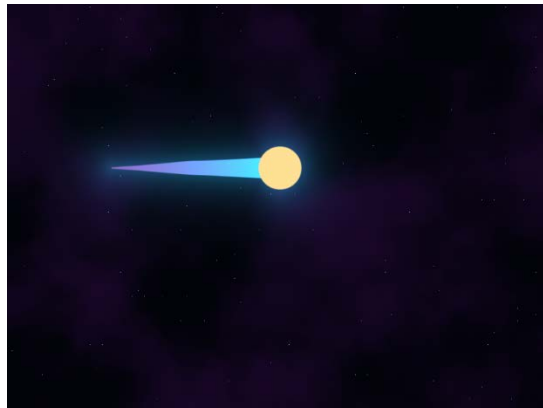
```
shaders      fullscreen.vert + mytoy.frag, through shadertoy.glsl
uniform      time 1.000 s, mode 0, octaves 5
gpu time     1.600 ms median, 1.966 ms p95
budget       2.000 ms at 800x600: PASS
uses noise.glsl yes
uses sdf_     yes
uses iMouse   yes
uses iMode    yes
TODO removed  yes
coverage     100.0% of the target is not black
frames       D:/Engineering/Year_III/Computer_Graphics/assignments/lab7/2110479-Computer-Grap
toy_1.png, toy_2.png
keys         0..3 debug modes, Up/Down octaves, S saves a png, Esc quits
watching     D:/Engineering/Year_III/Computer_Graphics/assignments/lab7/2110479-Computer-Grap
```

Write-ups

1. Your three frames, and a paragraph on how the effect works in maths: one sentence per term of the picture, naming the variable in your code.

Ans The three frames are

[t = 0]



[t = 1]



[t = 2]



The variable p converts $fragCoord$ into centered, aspect-corrected coordinates. The variable $center$ follows two sine curves driven by $iTime$, producing the comet's curved motion, but it is replaced by $mouseP$ while the mouse button is held. The variable $mist$ is four-octave fBM sampled at a slowly moving position and controls the nebula background and small tail distortion. The circle distance $coreD$ creates the pulsating comet head. The projection $tailPosition$ locates each pixel along $tailVector$, while $tailRadius$ decreases from 0.055 to 0.004 to create the tapered tail. The field $cometD$ is the minimum of $coreD$ and $tailD$, combining the head and tail. The masks convert the distance fields into antialiased shapes, while $glow$ uses exponential falloff outside the

comet. The variables starCell and starRandom produce sparse deterministic stars, and iTime changes their brightness to create twinkling.

2. The cost breakdown. Comment out your most expensive term, report the median before and after, and give it as a fraction of the frame.

Ans The most expensive term was the four-octave fbm() call used to calculate mist. With fBM enabled, the measured median was 1.719 ms. After commenting it out and replacing mist with 0.5, the median fell to 0.473 ms. The fBM term therefore cost approximately $1.719 - 0.473 = 1.246$ ms, which was $1.246 / 1.719 \times 100 = 72.5\%$ of the complete frame time.

3. One thing you changed to fit the budget, and what it cost you visually.

Ans I limited mist to four fBM octaves instead of five. The five-octave version measured 1.919 ms median, leaving only 0.081 ms below the 2.000 ms budget. Restoring four octaves reduced the final measurement to 1.752 ms and provided more reliable headroom. The visual cost is that the nebula and tail distortion contain slightly less fine-scale detail, but their large shapes and animation remain almost unchanged. The submitted shader has been restored to four octaves and passes the budget.

AI Prompt Log Overview

Part I

Prompt: Implement the circle shader and explain its coordinates and antialiasing.

Model: OpenAI Codex (GPT-5)

Idea: I needed help understanding gl_FragCoord, SDF circles, and fwidth.

Verification: Coverage was 15.9%, the center pixel was (42,77,255), and the image matched.

Part II

Prompt: Implement value noise and fBM and explain the performance results.

Model: OpenAI Codex (GPT-5)

Idea: I needed help interpolating lattice values and combining noise octaves.

Verification: The probes returned 169 and 84, and the image matched.

Part III

Prompt: Implement the SDF scene and explain its shape operations.

Model: OpenAI Codex (GPT-5)

Idea: I needed help with box SDFs, smooth union, and subtraction.

Verification: All distance probes passed, coverage was 12.5%, and the image matched.

Part IV

Prompt: Implement the vertex-stage heightmap and explain its performance.

Model: OpenAI Codex (GPT-5)

Idea: I needed help calculating terrain heights, normals, and lighting.

Verification: The normal probe was (149,244,81), and the image matched.

Part V

Prompt: Create an original animated ShaderToy using fBM, SDFs, mouse input, and debug modes.

Model: OpenAI Codex (GPT-5)

Idea: I wanted an animated comet that reused techniques from the earlier parts.

Verification: All requirements passed, and the final shader took 1.752 ms, below the 2.0 ms budget.

10:11 PM